

TP - gestion du cache

Prérequis : Un projet Laravel 12.x (018-tp-cache) ; Une table **events** avec au moins ces champs :

```
id, title, description, start_date, end_date, category, participants_count, created_at, updated_at
```

(Ajouter 15–30 enregistrements de test via factory ou seeder)

1 – Configuration de base & choix du driver Ouvre **.env** et choisis un driver simple pour commencer :

```
CACHE_STORE=file          # ou redis / database selon ce que tu as installé
```

Vérifier que ça fonctionne :

```
php artisan tinker
Cache::put('test', 'Hello cache!', 60);
Cache::get('test');
```

→ Tu devrais voir "Hello cache!"

2 – Première mise en cache naïve (put / get) Dans un contrôleur (ex: HomeController.php)

```
use Illuminate\Support\Facades\Cache;

public function index()
{
    $events = Cache::get('home.upcoming_events');

    if (!$events) {
        $events = Event::where('start_date', '>=', now())
            ->orderBy('participants_count', 'desc')
            ->take(8)
            ->get();

        Cache::put('home.upcoming_events', $events, 300); // 5 minutes
    }

    return view('welcome', compact('events'));
}
```

→ Problème : beaucoup de code verbeux → on va simplifier

3 – La méthode magique : remember Remplacer le code précédent par :

```
$events = Cache::remember(
    key: 'home.upcoming_events',
    ttl: now()->addMinutes(5),
    callback: fn() => Event::upcoming()->popular()->take(8)->get()
);
```

Créer un scope dans le model Event pour rendre ça lisible :

```
public function scopeUpcoming($query)
{
    return $query->where('start_date', '>=', now());
}

public function scopePopular($query)
```

```
{  
    return $query->orderByDesc('participants_count');  
}
```

4 – Cache permanent (rememberForever) Le nombre total de participants ne change que quand quelqu'un s'inscrit → on peut le garder plus longtemps.

```
$totalParticipants = Cache::rememberForever('stats.total_participants', function () {  
    return Event::sum('participants_count');  
});
```

5 – Invalidation manuelle du cache Ajouter une méthode dans un contrôleur admin ou dans un Listener quand un événement est créé/modifié/supprimé :

```
// Quand un événement est créé / modifié / supprimé  
Cache::forget('home.upcoming_events');  
Cache::forget('stats.total_participants');
```

Ou plus radical (mais à utiliser avec précaution) :

```
Cache::flush(); // tout le cache !
```

6 – Utilisation des tags (si tu passes sur redis ou memcached)

Changer le fichier `.env` → `CACHE_STORE=redis` (installer Redis + predis/predis si besoin)

Puis :

```
// Stockage  
Cache::tags(['home', 'stats', 'events'])->remember(  
    'home.upcoming_events',  
    now()->addMinutes(10),  
    fn() => Event::upcoming()->popular()->take(8)->get()  
);  
  
// Invalidation groupée très puissante  
Cache::tags(['events'])->flush(); // invalide tous les caches liés aux events
```

Exemple d'utilisation réelle :

- Quand un événement est ajouté → `Cache::tags('events')->flush();`
- Quand une catégorie change de nom → `Cache::tags('categories')->flush();`

7 – Éviter les race conditions avec withoutOverlapping Imaginons un **Observer** qui recalcule des statistiques lourdes après chaque changement :

```
php artisan make:observer EventObserver --model=Event
```

Implémenter l'observer :

```
<?php  
  
namespace App\Observers;  
  
use App\Models\Event;  
use Illuminate\Support\Facades\Cache;
```

```

class EventObserver
{
    public function saved(Event $event): void
    {
        // saved = créé OU mis à jour
        $this->invalidateRelatedCaches();
    }

    public function deleted(Event $event): void
    {
        $this->invalidateRelatedCaches();
    }

    private function invalidateRelatedCaches(): void
    {
        // Version minimale - très rapide
        Cache::forget('home.upcoming_events');
        Cache::forget('stats.total_participants');

        // Ou version plus large si tu utilises les tags (redis/memcached)
        // Cache::tags(['events', 'home', 'stats']->flush();

        // Option : si tu veux recalculer immédiatement (pas différé)
        // $this->refreshGlobalStatsCache();
    }

    // =====
    // Variante : recalcul synchrone direct (pas de job)
    // =====
    private function refreshGlobalStatsCache(): void
    {
        $stats = [
            'total_participants' => Event::sum('participants_count'),
            'most_active_category' => Event::groupBy('category')
                ->selectRaw('category, count(*) as count')
                ->orderByDesc('count')
                ->first(),
        ];

        Cache::tags(['stats']->forever('global_stats', $stats);
        Cache::forget('home.upcoming_events'); // souvent lié
    }
}

```

→ Enregistrer l'observer (méthode moderne Laravel 12)

Dans app/Models/Event.php :

```

use App\Observers\EventObserver;
use Illuminate\Database\Eloquent\Attributes\ObservedBy;

#[ObservedBy(EventObserver::class)]
class Event extends Model
{
    // ...
}

```

8 – Bonus : Cache::memo() (Laravel 12 – super pratique dans une même requête)

```

$statsA = Cache::memo()->get('global_stats'); // 1er appel → hit redis/file
$statsB = Cache::memo()->get('global_stats'); // → retour depuis la mémoire PHP (très rapide)

```

Très utile dans les middleware, les view composers, les resources, etc.

9 – Mini dashboard de debug cache

Créer une route **/debug/cache** :

```
Route::get('/debug/cache', function () {
    return [
        'driver'      => config('cache.default'),
        'upcoming'    => Cache::get('home.upcoming_events') ? 'exists' : 'missing',
        'total_users' => Cache::get('stats.total_participants') ?? 'missing',
        'keys_count'  => Cache::getRedis() ?->dbSize() ?? 'N/A (not redis)',
    ];
})->middleware('auth');
```